EC327: Lab 3

Data types, User Input,
Conditional Branches, Loops and Scope

Lab 2 Summary

- Previous Lab we looked at the following

  - Writing code in C++: `emacs lab2.cpp`
  - Piping output to another program using **|** `ls -la | less`
  - Redirecting output to a file using **>** `./a.out > output.txt`
  - Compiling C++ code using the GNU C++ compiler, `g++ -std=c++11 lab2.cpp -o lab2.exe`
  - Running the compiled program: `./a.out`
  - User input with `cin >> variable` and output with `cout << "Hello" << endl;`

Agenda

- Data types in C/C++
- Conditional branches
- Loops
- Scope

C/C++ Primitive Data Types

Type	Description	Range*
char	Common use is to store a single character ‘A’, ‘B’, ‘%’	signed: [-128 – 127] unsigned: [0 – 255]
int	Used to store whole numbers (integers)	signed: [-32,767, 32,767] unsigned: [0, 65,535]
long	Same as int but guaranteed to be 32 bits	signed: [−2,147,483,647, +2,147,483,647] unsigned: [0, 4,294,967,295]
float	Single precision floating point	1.2E-38 to 3.4E+38 – 6 decimal precision
double	Double precision floating point	2.3E-308 to 1.7E+308 – 15 decimal precision
bool	Used to store true or false	Bool is an integer too! However, range depends on platform

*Range depends on OS and compiler. These are minimum ranges.

Conditional Branches

Use conditional branches to make decisions

```
if (age >= 100)
  cout << "You are old\n";
else if (age >= 45)
  cout << "You are middle aged\n";
else
  cout << "You are young\n";
```

```
Enter your age: 100
You are old
Enter your age: 20
You are young
Enter your age: 55
You are middle aged
```

```
if (age >= 100)
  cout << "You are old\n";
if (age >= 45)
  cout << "You are middle aged\n";
else
  cout << "You are young\n";
```

```
Enter your age: 100
You are old
You are middle aged
Enter your age: 20
You are young
Enter your age: 55
You are middle aged
```

Conditional Branches

If expression 1 is true, it skips **else if** and **else**, then exit block.

If expression 1 is false, check expression 2. If expression 2 is true, do that, then exit block. If 2 is false, check 3. If expression 3 is true, do that, then exit block.

If 1, 2, 3 are all false, do **else**

```
if (expression 1) {
   // Executes when expression 1 is true
}
else if (expression 2) {
   // Executes when expression 2 is true
}
else if (expression 3) {
   // Executes when expression 3 is true
}
else {
   // Executes when none of the
   // above conditions are true.
}
```

Operators for Boolean Expressions

Relational operators

Operator	Meaning	Example
<	Less than	3 < 4 is **true** 3 < 2 is **false**
>	Greater than	4 > 3 is **true** 4 > 5 is **false**
<=	Less than or equal	3 <= 3 is **true** 1 <= 0 is **false**
>=	Greater than or equal	3 >= 3 is **true** 3 >= 4 is **false**
==	Equals	1 == 1 is **true** 1 == 2 is **false**
!=	Not Equals	1 != 0 is **true** 1 != 1 is **false**

Logical operators

Operator	Meaning	Example
&&	and	3 < 4 && 3 < 5 is **true** 2 < 3 && 1 < 0 is **false** true && false is **false**
||	or	4 < 3 || 3 < 5 is **true** 2 < 3 || 1 < 0 is **true** true || false is **true**
!	not	!false is **true** !true is **false** !true && !false is **false**

Loops - for

Execute statements multiple times

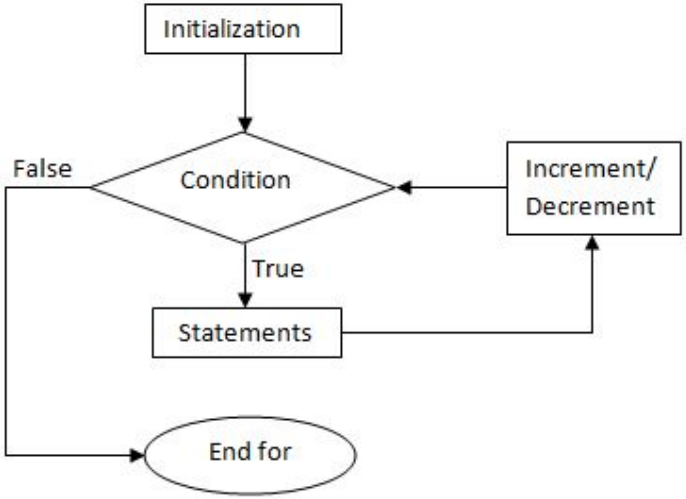


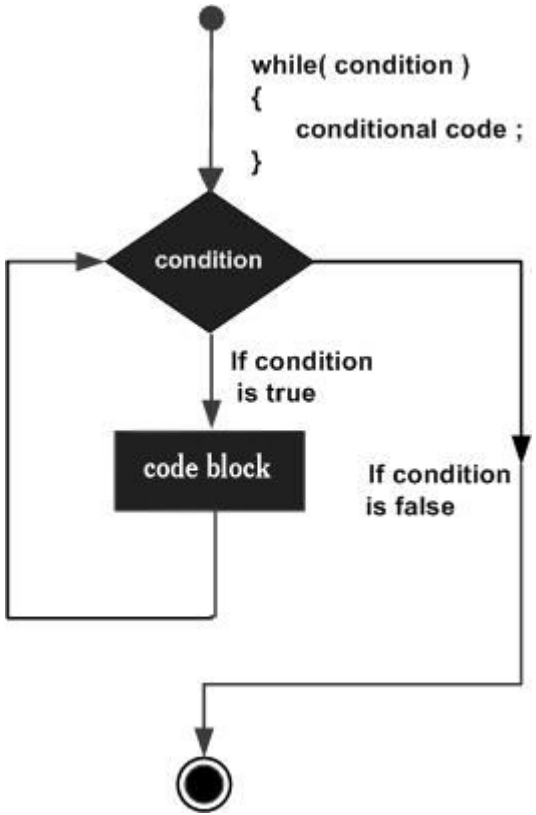
fig: Flowchart for for loop

```
// Syntax
for (init-statement; condition; iteration) {
  statements
}
// Example
for (int i = 0; i < 10; i++) {
  cout << i << ", ";
}
cout << endl;
return 0;
// Output
$ ./loop_example
0, 1, 2, 3, 4, 5, 6, 7, 8, 9,
```

Loops - while

Continue to do loop while the statement is true.

```
int i = 0;
while (i <= 5) {
  cout << i << ", ";
  i++;
}
// Output
$ ./while_example
0, 1, 2, 3, 4, 5,
```



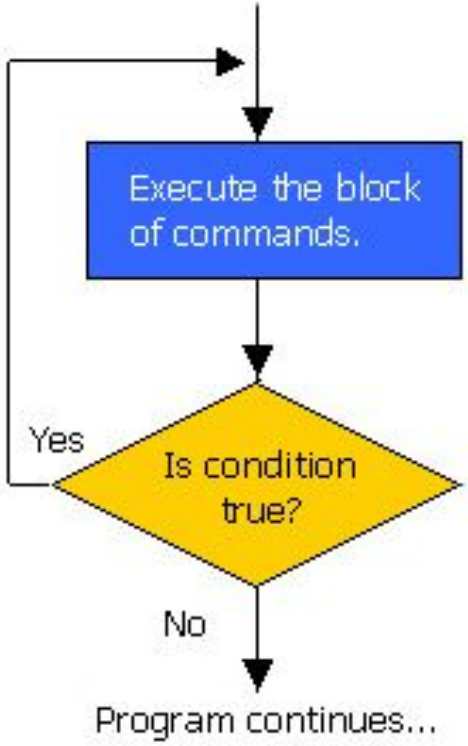
Loops – do while

Execute statements first **then** check condition

do…while ensures block is executed at least once

```
int i = 0;
do {
  cout << "i = ";
  cin >> i;
} while (i <= 5);
```

```
$./do_while
i = 0
i = -4
i = 3
i = 4
i = 5
i = 100
```



Variable Scope (1/2)

- Variables can only be accessed within their **scope**
- Local Scope: only available in block or function where it is defined
- Global: accessible everywhere; variable defined outside of functions and blocks

```
for (int count = 0; count < 10; count++) {
  cout << count << endl;
}
count = 20;
```

```
int size = 10;
if (size > 0) {
  int new_size = size*2;
  cout << "New size = "
       << new_size << endl;
}
cout << "New size again = "
     << new_size << endl;
```

Code will not compile! Variables are being accessed outside their scope.

Variable Scope (2/2)

- Be careful when defining variables
- Variables with the same name but different scope are **different**!
- Text editors and IDEs can help identify where variables are defined

```
int num_digits = 0;
int number = 2000;
if (number >= 0) {
  int num_digits = 0;
  while (number > 0) {
    number = number/10;
    num_digits = num_digits+1;
  }
}
cout << "Expected 4 but got "
     << num_digits << "!" << endl;
```

num_digits originally defined here

Variable Scope (2/2)

- Be careful when defining variables
- Variables with the same name but different scope are **different**!
- Text editors and IDEs can help identify where variables are defined

```
int num_digits = 0;
int number = 2000;
if (number >= 0) {
  int num_digits = 0;
  while (number > 0) {
    number = number/10;
    num_digits = num_digits+1;
  }
}
cout << "Expected 4 but got "
     << num_digits << "!" << endl;
```

num_digits originally defined here

num_digits defined in if statement scope

Variable Scope (2/2)

- Be careful when defining variables
- Variables with the same name but different scope are **different**!
- Text editors and IDEs can help identify where variables are defined

```
int num_digits = 0;
int number = 2000;
if (number >= 0) {
  int num_digits = 0;
  while (number > 0) {
    number = number/10;
    num_digits = num_digits+1;
  }
}
cout << "Expected 4 but got "
     << num_digits << "!" << endl;
```

num_digits originally defined here

num_digits defined in if statement scope

num_digits is 0 after leaving if statement scope

Coding Tips for Clean Code

- Always indent code blocks
- Choose good variable names

https://blog.usejournal.com/naming-your-variables-f9477ba002e9

- Group coding/space groups apart
- Use comments to help others understand complicated code
- Follow an established coding style guide (Google C++, LLVM, ISOCPP)

It makes helping easier and faster if we can read your code.

Questions?

```
public class Permuter                                   {
    private static void permute(int n, char[] a)        {
        if (n == 0)                                     {
            System.out.println(String.valueOf(a))       ;}
        else                                            {
            for (int i = 0; i <= n; i++)                {
                permute(n-1, a)                         ;
                swap(a, n % 2 == 0 ? i : 0, n)          ;}}}
    private static void swap(char[] a, int i, int j) {
        char saved = a[i]                               ;
        a[i] = a[j]                                     ;
        a[j] = saved                                    ;}}
```

Don’t do these…

```
//Collisions
static float collisionradius;
if(numplayers>1)
        for(k=0;k<numplayers;k++){
                for(i=k;i<numplayers;i++){
                        if(i==k)i++;
                        if(i<numplayers)
                                if((animation[player[i].targetanimation].attack!=reversed&&animation[player[i].targetan
                                        if((animation[player[i].currentanimation].attack!=reversed&&animation[player[i].
                                                if(player[i].howactive<=typesleeping&&player[k].howactive<=typesleeping
                                                        if(player[i].howactive!=typesittingwall&&player[k].howactive!=ty
                                                                if(i!=k&&player[i].whichpatchx==player[k].whichpatchx&&p
                                                                        if(player[i].coords.y>player[k].coords.y-3)
                                                                                if(player[i].coords.y<player[k].coords.y
                                                                                        if(player[i].coords.x>player[k].
                                                                                                if(player[i].coords.x<p
                                                                                                        if(player[i].coo
                                                                                                                if(playe
```